

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 5 (PBL - Part 1)

Name : V.Cherishma

Roll No.: A24126552126

This record contains four experiments carried out under Task 5. Each experiment is documented with its aim, objective, theory, algorithm, program, code explanation, test cases with valid and invalid inputs, output screenshots and result.

Q. No.	Question	CO
1	Interactive Product Calculator: Create a webpage containing a product name, quantity and price. Using JavaScript functions and event listeners, calculate the total price dynamically whenever the quantity or price changes. Display the result without refreshing the page.	CO-1
2	Node.js File Management: Create a Node.js application that accepts a filename and content from the user and performs create/write, read, append and display of the final contents using the File System module.	CO-2
3	Express.js Student Server: Create a basic Express.js server with routes /, /students and /about. The /students route should return a list of at least five students, while /about should display information about the application.	CO-2
4	Explain the Document Object Model (DOM). Demonstrate how JavaScript can dynamically change the content, style and attributes of an HTML element.	CO-1

EXPERIMENT 1

1. TITLE

Interactive Product Calculator Using JavaScript Functions, Event Listeners and DOM Updates.

2. AIM

To create a webpage with fields for the product name, quantity and price, and to calculate and display the total price dynamically, using JavaScript functions and event listeners, whenever the quantity or price changes, without refreshing the page.

3. OBJECTIVE

The objectives of this experiment are:

- To build an input form in HTML with fields for the product name, quantity and price, and to style it as a card using CSS.
- To select page elements from JavaScript using `document.getElementById()`.
- To read the values typed by the user with the `.value` property and convert them to numbers using `Number()`.
- To divide the program into small functions: `calculateTotal()` for the arithmetic, `validateInputs()` for checking the data and `updateCalculator()` for changing the page.
- To attach `updateCalculator()` to the input event of all three fields using `addEventListener()`, so that the total is recalculated as the user types.
- To validate the data and show an error message for negative values instead of displaying a wrong total.
- To update the page through the DOM using the `textContent` property and to format the amount with `toFixed(2)`.
- To stop the form from reloading the page by calling `event.preventDefault()` on its submit event.

4. THEORY

This program is event-driven. The HTML page is loaded only once; after that, every keystroke in the product name, quantity or price field fires an input event and the browser runs a JavaScript function in response. Because the result is written back into the page through the DOM, the document is never reloaded and the total appears to change instantly as the user types.

The script is divided into three functions, each with one job. `calculateTotal(quantity, price)` is a pure function: it only multiplies its two arguments and returns the answer, and it never touches the page. `validateInputs()` receives the raw text of the quantity and price fields, converts it with `Number()` and returns an object that says whether the data is valid, together with either the parsed numbers or an error message. `updateCalculator()` is the only function that changes the page; it calls the other two and writes the outcome into the result and error elements. Keeping calculation, validation and display separate makes each part easy to read and to test.

The `.value` property of an input always returns a string, even when the input type is number, so `Number()` is used to turn the text into a numeric value before it is multiplied. An empty field is detected first with `trim() === ""`, so that a field the user has not filled in yet is not treated as zero; in that case no error is shown and the total simply stays at 0.00. `Number.isNaN()` detects text that cannot be converted, and the numbers are then compared with 0 to reject negative values.

The submit event of the form is cancelled with `preventDefault()`. Without this, pressing Enter inside a field would submit the form and reload the page, discarding the calculation.

The key methods and concepts used in this program are:

- `document.getElementById("id")` returns the element whose id attribute matches, so JavaScript can read from it or write into it.
- `element.value` reads the text currently typed in an input field; it is always a string.
- `Number(text)` converts the string read from the input into a number so that arithmetic can be performed.
- `addEventListener("input", fn)` runs the function on every keystroke or spinner click; unlike "change", it does not wait for the field to lose focus. It is attached to all three fields with a `forEach()` loop.
- `trim()` removes leading and trailing spaces, so a blank value is detected as empty; it is also used to show a dash when the product name is blank.
- `Number.isNaN(value)` detects a value that could not be converted into a number.
- `element.textContent` writes plain text into an element, replacing what was there before; it is used for the product name, the total and the error message.
- `toFixed(2)` rounds the total to two decimal places and returns it as a string suitable for displaying an amount of money.
- `event.preventDefault()` stops the browser's default action, here the submission and reload of the form.

The basic flow of the program is:

The user types in any of the three fields → the input event fires → `updateCalculator()` runs → the raw values are passed to `validateInputs()` → if a field is empty, the total is reset to 0.00 without an error; if a value is negative or not a number, an error message is shown and the total is reset to 0.00 → otherwise the error is cleared, `calculateTotal()` multiplies the quantity by the price and the result is written into the page with two decimal places, along with the product name.

5. ALGORITHM / PROCEDURE

- Create three files, `index.html`, `style.css` and `script.js`, and link `style.css` in the head and `script.js` at the end of the body.
- In `index.html`, add a card containing a heading, a text input for the product name, number inputs for the quantity (`min="0", step="1"`) and the price (`min="0", step="0.01"`), an empty paragraph with `id="error-message"`, and a result section containing `id="result-product"` and `id="total-price"`.
- In `style.css`, style the page background, the card, the labels, the inputs, the red error message and the result box.
- In `script.js`, select the three inputs and the three output elements using `document.getElementById()`.
- Define `calculateTotal(quantity, price)` so that it returns `quantity * price`.
- Define `validateInputs(quantityRaw, priceRaw)` so that it converts both values with `Number()` and returns `{ valid: false, error: "" }` when a field is empty, an error message when a value is not a number or is negative, and `{ valid: true, quantity, price }` otherwise.

- Define `updateCalculator()` to read the three inputs and call `validateInputs()`; if the data is invalid, show the error and reset the total to 0.00; otherwise clear the error and write the total using `toFixed(2)`.
- Attach `updateCalculator()` to the input event of the product name, quantity and price fields using `forEach()` and `addEventListener()`.
- Add a submit listener to the form that calls `event.preventDefault()`.
- Save the files, open `index.html` in a browser and test the page with valid, empty, zero, negative and decimal values.

6. PROGRAM

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Interactive Product Calculator</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <main class="calculator-card">
    <h1>Product Total Calculator</h1>
    <p class="subtitle">Total updates instantly as you type – no page refresh needed.</p>

    <form id="calculator-form">
      <div class="field">
        <label for="product-name">Product Name</label>
        <input
          type="text"
          id="product-name"
          placeholder="e.g. Wireless Mouse"
          autocomplete="off"
        />
      </div>

      <div class="field-row">
        <div class="field">
          <label for="quantity">Quantity</label>
          <input
            type="number"
            id="quantity"
            min="0"
            step="1"
            placeholder="0"
          />
        </div>

        <div class="field">
          <label for="price">Price per unit (₹)</label>
          <input
            type="number"
            id="price"
            min="0"
            step="0.01"
            placeholder="0.00"
          />
        </div>
      </div>

      <p id="error-message" class="error-message" aria-live="polite"></p>
    </form>

    <section class="result" aria-live="polite">
      <span class="result-label">Total for <strong id="result-product"></strong></span>
```

```

        <span class="result-value">₹<span id="total-price">0.00</span></span>
    </section>
</main>

<script src="script.js"></script>
</body>
</html>

```

style.css

```

:root {
  --ink: #1c2b24;
  --paper: #f6f4ee;
  --accent: #2f6b4f;
  --accent-soft: #e4efe8;
  --border: #cdd6cf;
  --error: #b3413a;
  font-size: 16px;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  padding: 2rem 1rem;
  background: var(--paper);
  font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  color: var(--ink);
}

.calculator-card {
  width: 100%;
  max-width: 420px;
  background: #ffffff;
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 2rem;
}

h1 {
  margin: 0 0 0.4rem;
  font-size: 1.5rem;
  font-weight: 600;
}

.subtitle {
  margin: 0 0 1.5rem;
  color: #5b6b62;
  font-size: 0.9rem;
  line-height: 1.4;
}

.field {
  margin-bottom: 1rem;
  flex: 1;
}

.field-row {
  display: flex;
  gap: 1rem;
}

label {
  display: block;
  margin-bottom: 0.35rem;
  font-size: 0.85rem;
}

```

```

    font-weight: 600;
    color: #3c4a41;
}

input {
    width: 100%;
    padding: 0.6rem 0.7rem;
    border: 1px solid var(--border);
    border-radius: 6px;
    font-size: 1rem;
    font-family: inherit;
    color: var(--ink);
    background: #fdfdfb;
}

input:focus {
    outline: 2px solid var(--accent);
    outline-offset: 1px;
    border-color: var(--accent);
}

.error-message {
    min-height: 1.1rem;
    margin: 0.25rem 0 0;
    font-size: 0.85rem;
    color: var(--error);
}

.result {
    margin-top: 1.5rem;
    padding: 1rem 1.2rem;
    background: var(--accent-soft);
    border-radius: 8px;
    display: flex;
    align-items: baseline;
    justify-content: space-between;
    gap: 0.5rem;
}

.result-label {
    font-size: 0.9rem;
    color: #3c4a41;
}

.result-label strong {
    color: var(--ink);
}

.result-value {
    font-size: 1.4rem;
    font-weight: 700;
    color: var(--accent);
    white-space: nowrap;
}

@media (max-width: 420px) {
    .field-row {
        flex-direction: column;
        gap: 0;
    }
}

```

script.js

```

// DOM references
const productNameInput = document.getElementById("product-name");
const quantityInput = document.getElementById("quantity");
const priceInput = document.getElementById("price");

const resultProductEl = document.getElementById("result-product");
const totalPriceEl = document.getElementById("total-price");
const errorMessageEl = document.getElementById("error-message");

```

```
// Pure calculation logic (no DOM access here)

function calculateTotal(quantity, price) {
  return quantity * price;
}

/**
 * Validates the raw string inputs and returns parsed numbers,
 * or an error message if the inputs are invalid.
 */
function validateInputs(quantityRaw, priceRaw) {
  const quantity = Number(quantityRaw);
  const price = Number(priceRaw);

  if (quantityRaw.trim() === "" || priceRaw.trim() === "") {
    return { valid: false, error: "" }; // no error shown yet, just no result
  }
  if (Number.isNaN(quantity) || Number.isNaN(price)) {
    return { valid: false, error: "Quantity and price must be valid numbers." };
  }
  if (quantity < 0 || price < 0) {
    return { valid: false, error: "Quantity and price cannot be negative." };
  }
  return { valid: true, quantity, price };
}

//DOM update logic
function updateCalculator() {
  const productName = productNameInput.value.trim() || "-";
  const validation = validateInputs(quantityInput.value, priceInput.value);

  if (!validation.valid) {
    errorMessageEl.textContent = validation.error;
    totalPriceEl.textContent = "0.00";
    resultProductEl.textContent = productName;
    return;
  }

  errorMessageEl.textContent = "";
  const total = calculateTotal(validation.quantity, validation.price);

  resultProductEl.textContent = productName;
  totalPriceEl.textContent = total.toFixed(2);
}

//Event listener
[productNameInput, quantityInput, priceInput].forEach((input) => {
  input.addEventListener("input", updateCalculator);
});

// Prevent accidental form submission
document.getElementById("calculator-form").addEventListener("submit", (event) => {
  event.preventDefault();
});
```

7. CODE EXPLANATION

Code	Explanation
document.getElementById()	Selects the three input fields and the result, total and error elements by their id.
element.value	Reads the text currently typed into an input field as a string.
calculateTotal(quantity, price)	Pure function that returns quantity * price without touching the page.

Code	Explanation
validateInputs(quantityRaw, priceRaw)	Converts the raw text to numbers and returns { valid, quantity, price } or an error message.
quantityRaw.trim() === ""	Detects an empty field, so a blank input gives a total of 0.00 without showing an error.
Number.isNaN(quantity)	Detects text that cannot be converted into a number.
quantity < 0 price < 0	Rejects negative values with the message "Quantity and price cannot be negative."
updateCalculator()	The only function that changes the page; it validates the input and writes either the error or the total.
productNameInput.value.trim() "—"	Uses the product name if one has been typed, otherwise shows a dash.
textContent	Writes the product name, the total and the error message into the page without reloading it.
total.toFixed(2)	Rounds the total to two decimal places for currency display.
forEach(... addEventListener("input", updateCalculator))	Attaches the same function to the input event of all three fields.
event.preventDefault()	Cancels the form submission, which would otherwise reload the page.

8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested with valid inputs as well as invalid inputs (empty, zero and negative values). The results were recorded below.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open index.html in a browser	All fields are empty, no error message is shown, and "Total for —" with ₹0.00 is displayed	Default state displayed correctly	Pass
TC-02	Product: Wireless Mouse, Quantity: 2, Price: 750	Total updates to ₹1500.00 without reloading the page	Total for Wireless Mouse: ₹1500.00 displayed	Pass
TC-03	Change the quantity from 2 to 5 (price unchanged)	Total recalculates immediately to ₹3750.00	Total updated dynamically to ₹3750.00	Pass
TC-04	Clear the price field (invalid: empty)	Total resets to ₹0.00; no calculation is done and no wrong value is shown	Total reset to ₹0.00, no error message	Pass
TC-05	Quantity: 0, Price: 750 (zero quantity)	Zero is not negative, so it is accepted and the total is ₹0.00	₹0.00 displayed, no error message	Pass
TC-06	Quantity: -3, Price: 750 (invalid: negative quantity)	Error "Quantity and price cannot be negative." is shown and the total is ₹0.00	Error message displayed, total ₹0.00	Pass
TC-07	Quantity: 2, Price: -50 (invalid: negative price)	Error "Quantity and price cannot be negative." is shown and the total is ₹0.00	Error message displayed, total ₹0.00	Pass
TC-08	Product: USB Cable, Quantity: 3, Price: 99.99	Total is ₹299.97, rounded to two decimal places	Total for USB Cable: ₹299.97 displayed	Pass

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-09	After the error, enter a valid price of 120 with quantity 3	Error message disappears and ₹360.00 is displayed	Error cleared, total recalculated to ₹360.00	Pass
TC-10	Product name left blank, Quantity: 4, Price: 25	Total is still calculated and the product name is shown as a dash	"Total for —" with ₹100.00 displayed	Pass

9. OUTPUT

TC-01 – Page loaded with default values

The screenshot shows a web browser window titled "Interactive Product Calculator" with the URL "file:///tmp/task-05/01-product-calculator/index.html". The calculator interface has a title "Product Total Calculator" and a subtitle "Total updates instantly as you type — no page refresh needed." Below this, there are three input fields: "Product Name" with the placeholder text "e.g. Wireless Mouse", "Quantity" with the value "0", and "Price per unit (₹)" with the value "0.00". At the bottom, a green box displays "Total for —" followed by "₹0.00".

TC-02 – Valid product, quantity and price entered

The screenshot shows the same web browser window as before, but with updated input values. The "Product Name" field now contains "Wireless Mouse", the "Quantity" field contains "2", and the "Price per unit (₹)" field contains "750". The green box at the bottom now displays "Total for Wireless Mouse" followed by "₹1500.00".

TC-03 – Quantity changed, total recalculated without reload

The screenshot shows a web browser window titled "Interactive Product Calculator" with the address bar displaying "file:///tmp/task-05/01-product-calculator/index.html". The page content includes a form titled "Product Total Calculator" with the subtitle "Total updates instantly as you type — no page refresh needed." The form has three input fields: "Product Name" (containing "Wireless Mouse"), "Quantity" (containing "5"), and "Price per unit (₹)" (containing "750"). The "Quantity" field is highlighted with a green border. Below the inputs, a green button displays the total: "Total for Wireless Mouse ₹3750.00".

TC-04 – Invalid input: price field cleared

The screenshot shows the same web browser window as in TC-03. The "Product Name" field still contains "Wireless Mouse" and the "Quantity" field contains "5". The "Price per unit (₹)" field now contains "0.00" and is highlighted with a green border. The green button at the bottom now displays the total: "Total for Wireless Mouse ₹0.00".

TC-05 – Quantity is zero

The screenshot shows a web browser window titled "Interactive Product Calculator" with the URL "file:///tmp/task-05/01-product-calculator/index.html". The page displays a "Product Total Calculator" form. The "Product Name" field contains "Wireless Mouse". The "Quantity" field contains "0". The "Price per unit (₹)" field contains "750". Below the inputs, a green button displays "Total for Wireless Mouse ₹0.00".

Product Total Calculator
Total updates instantly as you type — no page refresh needed.

Product Name
Wireless Mouse

Quantity: 0 Price per unit (₹): 750

Total for Wireless Mouse ₹0.00

TC-06 – Invalid input: negative quantity

The screenshot shows the same web browser window as TC-05, but the "Quantity" field now contains "-3". A red error message "Quantity and price cannot be negative." is displayed below the input fields. The "Price per unit (₹)" field remains "750". The green button still displays "Total for Wireless Mouse ₹0.00".

Product Total Calculator
Total updates instantly as you type — no page refresh needed.

Product Name
Wireless Mouse

Quantity: -3 Price per unit (₹): 750

Quantity and price cannot be negative.

Total for Wireless Mouse ₹0.00

TC-07 – Invalid input: negative price

The screenshot shows a web browser window titled "Interactive Product Calculator" with the URL "file:///tmp/task-05/01-product-calculator/index.html". The calculator interface includes a title "Product Total Calculator", a subtitle "Total updates instantly as you type — no page refresh needed.", a "Product Name" input field containing "Wireless Mouse", a "Quantity" input field containing "2", and a "Price per unit (₹)" input field containing "-50". A red error message "Quantity and price cannot be negative." is displayed below the inputs. The total calculation at the bottom shows "Total for Wireless Mouse" as "₹0.00".

Product Total Calculator
Total updates instantly as you type — no page refresh needed.

Product Name
Wireless Mouse

Quantity: 2 Price per unit (₹): -50

Quantity and price cannot be negative.

Total for Wireless Mouse: ₹0.00

TC-08 – Decimal price rounded to two decimal places

The screenshot shows the same web browser window as TC-07. The calculator interface now has "USB Cable" in the "Product Name" field, "3" in the "Quantity" field, and "99.99" in the "Price per unit (₹)" field. The total calculation at the bottom shows "Total for USB Cable" as "₹299.97".

Product Total Calculator
Total updates instantly as you type — no page refresh needed.

Product Name
USB Cable

Quantity: 3 Price per unit (₹): 99.99

Total for USB Cable: ₹299.97

TC-09 – Valid value entered after an error, error message cleared

The screenshot shows a web browser window titled "Interactive Product Calculator" with the URL "file:///tmp/task-05/01-product-calculator/index.html". The page displays a "Product Total Calculator" form. The "Product Name" field contains "USB Cable". The "Quantity" field contains "3". The "Price per unit (₹)" field contains "120". The total calculated is "₹360.00".

Field	Value
Product Name	USB Cable
Quantity	3
Price per unit (₹)	120
Total	₹360.00

TC-10 – Product name left blank

The screenshot shows the same web browser window. The "Product Name" field now contains "e.g. Wireless Mouse". The "Quantity" field contains "4". The "Price per unit (₹)" field contains "25". The total calculated is "₹100.00".

Field	Value
Product Name	e.g. Wireless Mouse
Quantity	4
Price per unit (₹)	25
Total	₹100.00

10. RESULT

Thus, an interactive product calculator was successfully built using JavaScript functions, event listeners and DOM updates. The total was calculated and displayed instantly whenever the product name, quantity or price changed, and the page was never reloaded. Negative values were rejected with a red error message and the total was reset to ₹0.00, while an empty field simply left the total at ₹0.00. Two points were noted during testing: a quantity of 0 is accepted and gives ₹0.00 because only negative values are rejected, and the product name is not validated, so a total is still calculated when it is left blank. Checking that the quantity is at least 1 and that the product name is not empty would make the validation stricter.

EXPERIMENT 2

1. TITLE

Node.js File Management Application Using the File System (fs) Module.

2. AIM

To create a Node.js application that accepts a filename and content from the user and, using the File System module, creates and writes the file, reads it, appends more content to it and displays the final contents.

3. OBJECTIVE

The objectives of this experiment are:

- To take input from the terminal using the built-in readline module.
- To read the lines typed by the user with an async iterator and await, so that the questions are asked one after another.
- To create and write a file using `fs.writeFileSync()`.
- To read the contents of a file using `fs.readFileSync()` with utf8 encoding.
- To add content to the end of an existing file using `fs.appendFileSync()`.
- To display the final contents of the file after the append operation.
- To keep the file operations in a separate module and export them using `module.exports`.
- To store all files in a dedicated files folder that is created automatically, using `fs.existsSync()`, `fs.mkdirSync()`, `path.join()` and `__dirname`.
- To catch errors such as an invalid path with a catch block and report them without crashing the program.

4. THEORY

Node.js allows JavaScript to run outside the browser, on the server side. The File System module, imported with `require("fs")`, gives JavaScript direct access to the files on the machine, which is not possible from a web page.

The fs module offers an asynchronous and a synchronous version of every operation. This program uses the synchronous versions: `writeFileSync()`, `readFileSync()` and `appendFileSync()`. A synchronous method waits until the disk operation has finished and then returns its result, or throws an error if it fails. This lets the steps be written in the order in which they must happen (write, read, append, read again) without nested callbacks, which is suitable here because the files are tiny and the program does only one thing at a time.

Terminal input is handled by the readline module. The interface is created with `process.stdin` and `process.stdout`, and `rl[Symbol.asyncIterator]()` gives an iterator whose `next()` method resolves with the next line the user types. The helper function `ask(question)` prints the prompt with `process.stdout.write()` and awaits `lines.next()`. Because `main()` is an async function, each await pauses the program until the user presses Enter, so the prompts appear in sequence.

The program is split into two files. `fileoperations.js` holds four helper functions, `getFilePath()`, `createAndWrite()`, `readFile()` and `appendToFile()`, and exports them with `module.exports`. `app.js` imports them with `require("./fileoperations")` and uses them in order. All the files are kept in a folder

named files, whose location is built with `path.join(__dirname, "files")` so that it works whichever directory the program is started from; `ensureDataDir()` creates the folder the first time it is needed.

The synchronous `fs` methods throw an exception when something goes wrong, for example `ENOENT` when a folder in the path does not exist, or `EISDIR` when the path points to a directory instead of a file. The call `main().catch()` receives such an error, prints `err.message`, closes the readline interface and sets `process.exitCode` to 1 to show that the program ended with a failure.

The key methods and concepts used in this program are:

- `require("fs")`, `require("path")` and `require("readline")` import the built-in File System, Path and readline modules.
- `readline.createInterface({ input: process.stdin, output: process.stdout })` connects the program to the keyboard and the screen.
- `rl[Symbol.asyncIterator]()` and `lines.next()` wait for the next line typed by the user; `value ?? ""` returns an empty string if no input is available.
- `fs.existsSync(path)` and `fs.mkdirSync(path)` check for the files folder and create it if it is missing.
- `fs.writeFileSync(path, content, "utf8")` creates the file, or replaces its contents if it already exists.
- `fs.readFileSync(path, "utf8")` reads the file and returns its contents as a string; without `"utf8"` a Buffer is returned.
- `fs.appendFileSync(path, text, "utf8")` adds text to the end of the file without deleting what is already there; `"\n"` is placed first so that the new content starts on a new line.
- `path.join(__dirname, "files", filename)` builds a full file path that works on any operating system.
- `module.exports` exposes the four functions so that `app.js` can import them.
- `async / await` lets the asynchronous prompts be written as ordinary top-to-bottom code.
- `err.message` prints a readable description of a failure, and `process.exitCode = 1` marks the program as failed.
- `rl.close()` ends the readline interface so that the program can exit.

The basic flow of the program is:

The program asks for a filename → asks for the content → the files folder is created if it does not exist → the file is created and written → the file is read and its contents are printed → the program asks for additional content → the content is appended on a new line → the file is read again and the final contents are printed → the readline interface is closed. If any step throws an error, the catch block prints the message and the program ends.

5. ALGORITHM / PROCEDURE

- Create a project folder, initialise it with `npm init -y`, and create two files, `fileoperations.js` and `app.js`.
- In `fileoperations.js`, import `fs` and `path`, and define `DATA_DIR` as `path.join(__dirname, "files")`.
- Define `ensureDataDir()` so that it creates the files folder if it does not exist.
- Define `getFilePath(filename)` that returns the full path of the file inside the files folder.
- Define `createAndWrite(filename, content)` that calls `ensureDataDir()` and writes the file using `fs.writeFileSync()`.

- Define `readFile(filename)` that returns `fs.readFileSync(filePath, "utf8")`.
- Define `appendToFile(filename, extraContent)` that calls `fs.appendFileSync()` with `"\n"` placed before the new text.
- Export `getFilePath`, `createAndWrite`, `readFile` and `appendToFile` using `module.exports`.
- In `app.js`, import `readline` and the file operations, create the `readline` interface and its `async` iterator, and define `ask(question)` that prints a prompt and waits for the next line.
- In `main()`, ask for the filename (trimmed) and the content, call `createAndWrite()` and print the file path.
- Call `readFile()` and print the contents, ask for the additional content, and call `appendToFile()`.
- Call `readFile()` once more to print the final contents, then close the `readline` interface.
- Call `main().catch()` to print any error message, close `readline` and set `process.exitCode = 1`.
- Run the program with `node app.js` and test it with a valid filename, an empty filename, a folder that does not exist, an existing file and empty content.

6. PROGRAM

fileoperations.js

```
const fs = require("fs");
const path = require("path");

// All files this app manages are kept inside a dedicated "files" folder.
const DATA_DIR = path.join(__dirname, "files");

/** Makes sure the "files" folder exists before we try to use it. */
function ensureDataDir() {
  if (!fs.existsSync(DATA_DIR)) {
    fs.mkdirSync(DATA_DIR);
  }
}

/** Resolves a filename to its full path inside the data folder. */
function getFilePath(filename) {
  return path.join(DATA_DIR, filename);
}

/** Creates the file (or overwrites it) with the given content. */
function createAndWrite(filename, content) {
  ensureDataDir();
  const filePath = getFilePath(filename);
  fs.writeFileSync(filePath, content, "utf8");
  return filePath;
}

/** Reads and returns the file's current contents. */
function readFile(filename) {
  const filePath = getFilePath(filename);
  return fs.readFileSync(filePath, "utf8");
}

/** Appends additional content to the end of the file, on a new line. */
function appendToFile(filename, extraContent) {
  const filePath = getFilePath(filename);
  fs.appendFileSync(filePath, "\n" + extraContent, "utf8");
  return filePath;
}

module.exports = {
  getFilePath,
  createAndWrite,
  readFile,
  appendToFile,
}
```

```
};
```

app.js

```
const readline = require("readline");
const fileOps = require("./fileoperations");

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});
const lines = rl[Symbol.asyncIterator]();

/** Prompts the user and waits for the next line of input. */
async function ask(question) {
  process.stdout.write(question);
  const { value } = await lines.next();
  return value ?? "";
}

async function main() {
  console.log("=== Node.js File Management ===\n");

  const filename = (await ask("Enter a filename (e.g. notes.txt): ")).trim();
  const initialContent = await ask("Enter the content to write to the file: ");

  // 1. Create/write
  const filePath = fileOps.createAndWrite(filename, initialContent);
  console.log(`\n✔ File created and written: ${filePath}`);

  // 2. Read
  console.log("\n--- Contents after writing ---");
  console.log(fileOps.readFile(filename));
  console.log("--- end of file ---");

  // 3. Append
  const extraContent = await ask("\nEnter additional content to append: ");
  fileOps.appendToFile(filename, extraContent);
  console.log(`\n✔ Content appended to: ${filePath}`);

  // 4. Display final contents
  console.log("\n--- Final contents after appending ---");
  console.log(fileOps.readFile(filename));
  console.log("--- end of file ---");

  rl.close();
}

main().catch((err) => {
  console.error("Something went wrong:", err.message);
  rl.close();
  process.exitCode = 1;
});
```

package.json

```
{
  "name": "node-file-management",
  "version": "1.0.0",
  "description": "Node.js File System module demo - create/write, read, append (PBL Q2)",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  }
}
```

7. CODE EXPLANATION

Code	Explanation
<code>require("fs") / require("path")</code>	Imports the built-in File System and Path modules.
<code>path.join(__dirname, "files")</code>	Builds the location of the files folder next to the program, independent of where the program is started.
<code>fs.existsSync() / fs.mkdirSync()</code>	Checks whether the files folder exists and creates it when it does not.
<code>fs.writeFileSync(filePath, content, "utf8")</code>	Creates the file with the given content, or replaces it if it already exists.
<code>fs.readFileSync(filePath, "utf8")</code>	Reads the file and returns its contents as a readable string instead of a Buffer.
<code>fs.appendFileSync(filePath, "\n" + extraContent, "utf8")</code>	Adds the new content to the end of the file on a new line, keeping the old content.
<code>module.exports</code>	Exports <code>getFilePath</code> , <code>createAndWrite</code> , <code>readFile</code> and <code>appendToFile</code> so that <code>app.js</code> can import them.
<code>readline.createInterface()</code>	Connects the program to the keyboard (<code>process.stdin</code>) and the screen (<code>process.stdout</code>).
<code>rl[Symbol.asyncIterator]() / lines.next()</code>	Provides a way to wait for the next line typed by the user.
<code>async function ask(question)</code>	Prints the prompt and returns the user's answer; value <code>?? ""</code> gives an empty string if no input is available.
<code>await</code>	Pauses <code>main()</code> until the user has typed an answer, so the prompts appear one after another.
<code>main().catch(...)</code>	Catches any error thrown by the file operations, prints <code>err.message</code> and sets <code>process.exitCode = 1</code> .
<code>rl.close()</code>	Closes the readline interface so that the program ends.

8. TEST CASES AND EXECUTION DATA

The program was run from the terminal using `node app.js` and tested with valid input as well as invalid input such as an empty filename and a path that does not exist. The results were recorded below.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Filename: student.txt; content: "This file is created using Node.js File System module."; append: "This line was appended later to demonstrate the fs.appendFileSync() method."	File is created inside the files folder, read, appended to, and the final contents show both lines	All four operations completed in order; both lines displayed	Pass
TC-02	Press Enter without typing a filename (invalid: empty); content: "Some content"	No file is created; the error is caught and printed instead of crashing the program	The path resolved to the files folder itself; "Something went wrong: EISDIR ..." displayed and the program ended	Pass

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-03	Filename entered as spaces only (invalid)	trim() reduces it to an empty string, so the same error as TC-02 is shown	Same EISDIR error message displayed	Pass
TC-04	Filename: nofolder/data.txt (invalid: folder does not exist)	An ENOENT error is printed and no file is created	"Something went wrong: ENOENT: no such file or directory ..." displayed	Pass
TC-05	Re-run with the existing filename student.txt; content: "Overwritten content."; append: "Appended again."	The old content is replaced by the new content, then the appended line is added	File overwritten; final contents are the two new lines	Pass
TC-06	Filename: empty.txt; content left blank; append: "Now it has content."	An empty file is created and the appended text is added on a new line	Empty file created; final contents show a blank first line followed by the appended text	Pass

9. OUTPUT

TC-01 – Valid filename and content: create, read, append and final display

```

Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt): student.txt
Enter the content to write to the file: This file is created using Node.js File System module.

✅ File created and written: /tmp/task-05/02-node-file-management/files/student.txt

--- Contents after writing ---
This file is created using Node.js File System module.
--- end of file ---

Enter additional content to append: This line was appended later to demonstrate the fs.appendFileSync() method.

✅ Content appended to: /tmp/task-05/02-node-file-management/files/student.txt

--- Final contents after appending ---
This file is created using Node.js File System module.
This line was appended later to demonstrate the fs.appendFileSync() method.
--- end of file ---

02-node-file-management>

```

TC-02 – Invalid input: filename left empty

```

Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt):
Enter the content to write to the file: Some content
Something went wrong: EISDIR: illegal operation on a directory, open '/tmp/task-05/02-node-file-management/files'

02-node-file-management>

```

TC-03 – Invalid input: filename containing only spaces

```
Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt):
Enter the content to write to the file: Some content
Something went wrong: EISDIR: illegal operation on a directory, open '/tmp/task-05/02-node-file-management/files'

02-node-file-management>
```

TC-04 – Invalid input: filename inside a folder that does not exist

```
Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt): nofolder/data.txt
Enter the content to write to the file: Some content
Something went wrong: ENOENT: no such file or directory, open '/tmp/task-05/02-node-file-management/files/nofolder/data.txt'

02-node-file-management>
```

TC-05 – Existing file is overwritten and then appended to

```
Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt): student.txt
Enter the content to write to the file: Overwritten content.

✅ File created and written: /tmp/task-05/02-node-file-management/files/student.txt

--- Contents after writing ---
Overwritten content.
--- end of file ---

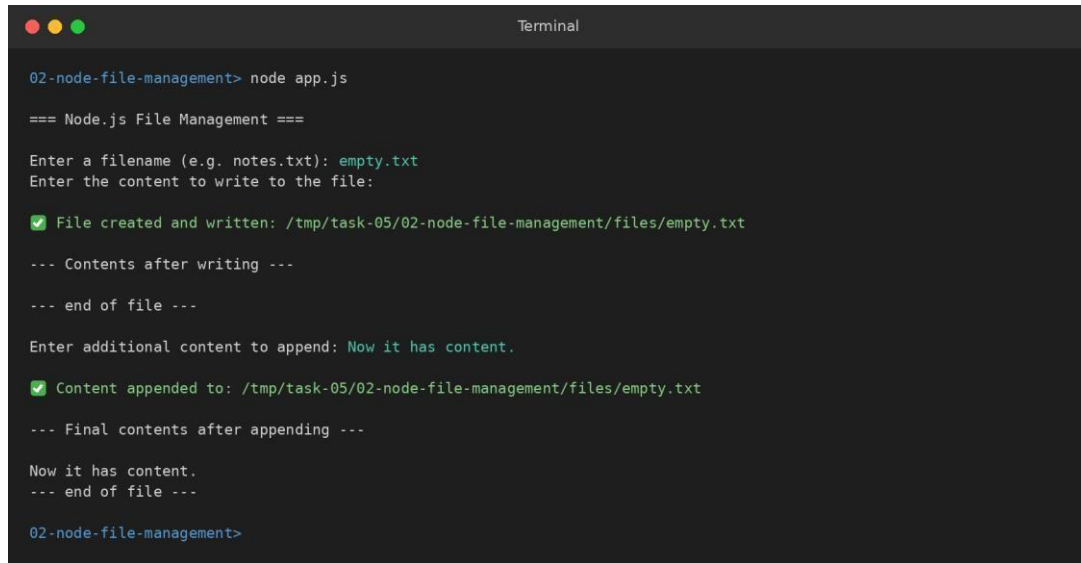
Enter additional content to append: Appended again.

✅ Content appended to: /tmp/task-05/02-node-file-management/files/student.txt

--- Final contents after appending ---
Overwritten content.
Appended again.
--- end of file ---

02-node-file-management>
```

TC-06 – Empty initial content accepted, append adds the first line



```
Terminal

02-node-file-management> node app.js

=== Node.js File Management ===

Enter a filename (e.g. notes.txt): empty.txt
Enter the content to write to the file:

✅ File created and written: /tmp/task-05/02-node-file-management/files/empty.txt

--- Contents after writing ---

--- end of file ---

Enter additional content to append: Now it has content.

✅ Content appended to: /tmp/task-05/02-node-file-management/files/empty.txt

--- Final contents after appending ---

Now it has content.
--- end of file ---

02-node-file-management>
```

10. RESULT

Thus, a Node.js file management application was successfully created using the File System module. The program accepted a filename and content from the user, created and wrote the file inside the files folder, read and displayed it, appended additional content and displayed the final contents. Errors caused by invalid paths, such as a folder that does not exist, were caught by the catch block and printed instead of crashing the program. Two points were noted during testing: an empty filename is not checked separately, so it fails only when the write is attempted, with an EISDIR error, and `writeFileSync()` silently replaces an existing file, so running the program twice with the same filename loses the earlier content. Validating the filename before writing and asking for confirmation before overwriting would make the application safer.

EXPERIMENT 3

1. TITLE

Express.js Student Server with Routing and JSON Responses.

2. AIM

To create a basic Express.js server with the routes `/`, `/students` and `/about`, where `/students` returns a list of five students and `/about` displays information about the application.

3. OBJECTIVE

The objectives of this experiment are:

- To create a Node.js project and install Express.js using npm.
- To import Express with `require("express")`, create the application object and start a server on port 3000 with `app.listen()`.
- To keep the student data in an in-memory array of objects.
- To define GET routes for `/`, `/students` and `/about` using `app.get()`.
- To send a plain text response using `res.send()`.
- To send the student list and the application information as JSON using `res.json()`.
- To generate a value dynamically, such as `totalStudents`, from `students.length`.
- To test every route in the browser and with curl, including an undefined route and an undefined HTTP method.

4. THEORY

Express.js is a web framework built on top of the Node.js http module. Instead of parsing the URL and the request method by hand, the developer describes the server as a set of routes. A route is a combination of an HTTP method and a path; when a request arrives, Express compares it with the routes in the order in which they were defined and runs the handler of the first one that matches.

Every route handler receives two objects: the request object `req`, which describes what the client asked for, and the response object `res`, which is used to send the answer back. Exactly one response must be sent for each request. `res.send()` sends text or HTML and sets the Content-Type header to `text/html`, whereas `res.json()` converts a JavaScript object or array into JSON text and sets the Content-Type header to `application/json`, which is why it is used for the student list.

The student data is kept in an array of objects inside the program, which acts as a very small in-memory database. Each student has an id, a name, a branch and a section. The `/about` route returns a JSON object containing the application name, a description, the list of available routes and `totalStudents`, which is calculated from `students.length` so that the count stays correct if students are added to the array.

This program does not define a handler for unknown routes. When no route matches a request, Express's built-in final handler answers with the status 404 and a short message such as "Cannot GET /teachers". The same happens when a route exists but not for that HTTP method: only GET is defined for `/students`, so a POST request to it also receives a 404 response.

The key methods and concepts used in this program are:

- `require("express")` imports the Express framework, which must first be installed with `npm install express`.
- `const app = express()` creates the application object that holds all the routes.
- `app.get(path, handler)` registers a handler for GET requests to a path, which is the method a browser uses when a URL is typed in the address bar.
- `res.send(text)` sends a plain text or HTML response to the client.
- `res.json(data)` sends an array or object as JSON and sets the correct content type automatically.
- `students.length` gives the number of elements in the array, used for `totalStudents`.
- `app.listen(PORT, callback)` starts the server and runs the callback once it is listening; a template literal is used to print the address with the port number.
- `package.json` records the express dependency and a "start" script so that the server can be started with `npm start`.

The basic flow of the program is:

The server is started with `node server.js` → Express listens on port 3000 → a browser or curl sends a GET request → Express compares the path with the registered routes → `/` returns the welcome text, `/students` returns the JSON array and `/about` returns the JSON description → a path or method that matches no route receives Express's default 404 response.

5. ALGORITHM / PROCEDURE

- Create a project folder and run `npm init -y` to generate `package.json`.
- Install Express using `npm install express`.
- Create `server.js`, import Express with `require("express")`, create the app object and store the port number 3000 in a constant.
- Define an array named `students` containing five objects, each with `id`, `name`, `branch` and `section`.
- Define a GET route for `/` that sends the welcome message using `res.send()`.
- Define a GET route for `/students` that returns the `students` array using `res.json()`.
- Define a GET route for `/about` that returns a JSON object with `appName`, `description`, the list of routes and `totalStudents` (`students.length`).
- Call `app.listen(PORT, callback)` and print the server address from the callback.
- Add a "start" script to `package.json` and start the server with `npm start` or `node server.js`.
- Open `http://localhost:3000/`, `/students` and `/about` in the browser and confirm each response.
- Test an undefined route (`/teachers`) and use curl to check the status code and headers of GET `/students` and the response to a POST request to `/students`.

6. PROGRAM

server.js

```
const express = require("express");
const app = express();
const PORT = 3000;

// In-memory "database" of students.
const students = [
  { id: 1, name: "Saranya", branch: "CSM", section: "B" },
  { id: 2, name: "Arjun Rao", branch: "CSE", section: "A" },
```

```

    { id: 3, name: "Priya Sharma", branch: "CSM", section: "B" },
    { id: 4, name: "Kiran Kumar", branch: "IT", section: "C" },
    { id: 5, name: "Divya Reddy", branch: "CSM", section: "A" },
  ];

  // GET / -> simple welcome route
  app.get("/", (req, res) => {
    res.send("Welcome to the Student Server. Try /students or /about.");
  });

  // GET /students -> return the full list of students as JSON
  app.get("/students", (req, res) => {
    res.json(students);
  });

  // GET /about -> information about this application
  app.get("/about", (req, res) => {
    res.json({
      appName: "Student Server",
      description:
        "A basic Express.js REST-style server built for the Full Stack Web Development PBL.",
      routes: ["/", "/students", "/about"],
      totalStudents: students.length,
    });
  });

  app.listen(PORT, () => {
    console.log(`Student server running at http://localhost:${PORT}`);
  });

```

package.json

```

{
  "name": "express-student-server",
  "version": "1.0.0",
  "description": "Basic Express.js server with /, /students and /about routes (PBL Q3)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  }
}

```

7. CODE EXPLANATION

Code	Explanation
<code>require("express")</code>	Imports the Express framework installed through npm.
<code>const app = express()</code>	Creates the Express application object on which the routes are registered.
<code>const PORT = 3000</code>	Stores the port number on which the server listens.
<code>const students = [...]</code>	In-memory array of five student objects, each with id, name, branch and section.
<code>app.get("/", handler)</code>	Handles GET requests to the home route.
<code>res.send("...")</code>	Sends a plain text response to the browser.
<code>app.get("/students", handler)</code>	Handles GET requests to /students.
<code>res.json(students)</code>	Converts the array into JSON and sets the Content-Type header to application/json.
<code>app.get("/about", handler)</code>	Handles GET requests to /about and returns a JSON object describing the application.

Code	Explanation
totalStudents: students.length	Calculates the number of students from the array instead of writing a fixed number.
app.listen(PORT, callback)	Starts the server and prints its address once it is listening.
"start": "node server.js"	npm script that runs the server when npm start is typed (package.json).
"express": "^4.19.2"	Dependency entry that records the Express version the project needs (package.json).

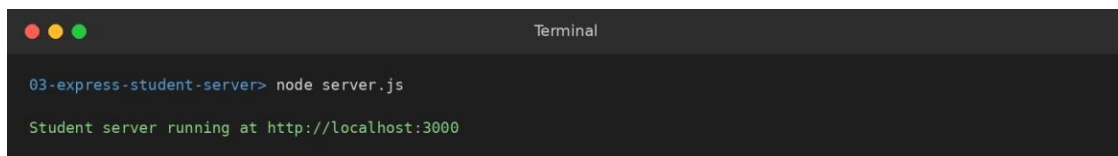
8. TEST CASES AND EXECUTION DATA

The server was started from the terminal and each route was verified in the browser and with curl, including a route that does not exist and an HTTP method that is not defined. The results were recorded below.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Run node server.js in the terminal	The message "Student server running at http://localhost:3000" is printed	Server started on port 3000	Pass
TC-02	Open http://localhost:3000/ in a browser	The text "Welcome to the Student Server. Try /students or /about." is displayed	Welcome message displayed	Pass
TC-03	Open http://localhost:3000/students	A JSON array of five students with id, name, branch and section is returned	JSON list of five students displayed	Pass
TC-04	Open http://localhost:3000/about	A JSON object with appName, description, routes and totalStudents (5) is returned	JSON object displayed; totalStudents is 5	Pass
TC-05	Open http://localhost:3000/teachers (invalid route)	No route matches, so Express returns a 404 response "Cannot GET /teachers"	404 response with "Cannot GET /teachers" displayed	Pass
TC-06	Run curl -i http://localhost:3000/students	Status 200 OK, Content-Type application/json and the student list in the body	HTTP/1.1 200 OK with JSON body received	Pass
TC-07	Run curl -i -X POST http://localhost:3000/students (method not defined)	The request matches no route and returns 404 Not Found	HTTP/1.1 404 Not Found with "Cannot POST /students" returned	Pass

9. OUTPUT

TC-01 – Server started from the terminal



```

03-express-student-server> node server.js

Student server running at http://localhost:3000

```

TC-02 – Home route / in the browser



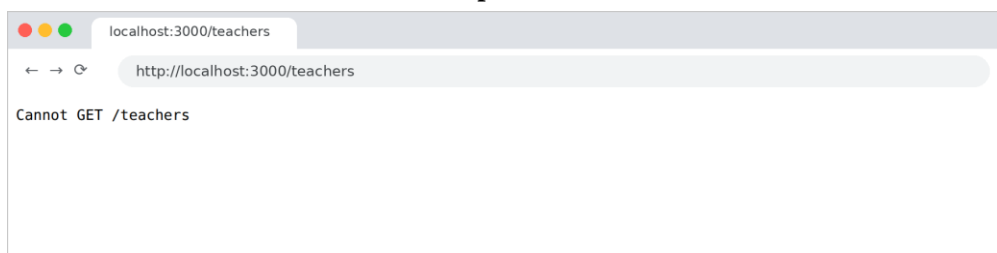
TC-03 – /students route returning the JSON student list



TC-04 – /about route returning the application information



TC-05 – Undefined route /teachers returns Express's default 404



TC-06 – GET /students checked with curl (200 OK)

```
Terminal

03-express-student-server> curl -i http://localhost:3000/students

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 290
ETag: W/"122-ahcbosZye6yBg/zEGGK9933ee14"
Date: Sun, 20 Sep 2026 07:10:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"name":"Saranya","branch":"CSM","section":"B"},{"id":2,"name":"Arjun Rao","branch":"CSE","section":"A"},{"id":3,"name":"Priya Sharma","branch":"CSM","section":"B"},{"id":4,"name":"Kiran Kumar","branch":"IT","section":"C"},{"id":5,"name":"Divya Reddy","branch":"CSM","section":"A"}]
```

TC-07 – POST /students returns 404 because only GET is defined

```
Terminal

03-express-student-server> curl -i -X POST http://localhost:3000/students

HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 148
Date: Sun, 20 Sep 2026 07:10:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot POST /students</pre>
</body>
</html>
```

10. RESULT

Thus, a basic Express.js student server was successfully created with the routes /, /students and /about. The home route returned plain text using `res.send()`, while /students and /about returned JSON using `res.json()`, and every route was verified in the browser and with curl. Requests to an undefined route, or with an HTTP method that is not defined, received Express's default 404 response. The server keeps its data in memory, so the student list is reset each time the server restarts. Adding a custom 404 handler, POST, PUT and DELETE routes and a database would allow students to be added, updated and removed permanently.

EXPERIMENT 4

1. TITLE

Demonstration of the Document Object Model (DOM): Changing the Content, Style and Attributes of an HTML Element Using JavaScript.

2. AIM

To explain the Document Object Model and to demonstrate how JavaScript can dynamically change the content, the style and the attributes of an HTML element at runtime.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how the browser represents an HTML page as a tree of nodes called the Document Object Model.
- To access elements on the page from JavaScript using `document.getElementById()`.
- To change the visible text of an element using the `textContent` property.
- To change the appearance of an element at runtime using the `style` property (color, `backgroundColor`, padding and `borderRadius`).
- To add or modify an attribute of an element using `setAttribute()`.
- To run all of these changes from a single click event handler attached with `addEventListener()`.
- To verify the changes visually in the browser and by inspecting the element in the browser console.

4. THEORY

When a browser loads an HTML page it does not keep the file as plain text. It parses the file and builds an object-based model of it in memory called the Document Object Model, or DOM. Every element, attribute and piece of text becomes a node, and the nodes are arranged as a tree: the document object is the root, `<html>` is below it, then `<head>` and `<body>`, and so on down to the individual elements.

The DOM is a live model. It is not a copy of the HTML file but what the browser actually displays: whenever JavaScript changes a node, the browser immediately repaints the page to match, so the page can change without being reloaded and without the HTML file being edited. Reloading the page rebuilds the DOM from the file, so all such changes are lost.

JavaScript can change three different things about an element, and this program demonstrates all three:

- Content – the text inside the element, changed here with the `textContent` property, which replaces whatever text the element held.
- Style – the appearance of the element, changed here through the `style` object. CSS properties written with a hyphen become camelCase in JavaScript, so `background-color` is written as `backgroundColor` and `border-radius` as `borderRadius`.
- Attributes – the extra information carried by the tag, changed here with `setAttribute("title", ...)`, which adds a title attribute that the browser shows as a tooltip when the mouse rests on the heading.

The changes are made inside a function passed to `addEventListener("click", ...)`, so nothing happens until the user clicks the button. This is the event-driven model on which interactive web pages are built.

The key methods and concepts used in this program are:

- `document.getElementById("id")` returns the node in the DOM tree whose id attribute matches.
- `addEventListener("click", fn)` registers a function to run when the button is clicked.
- `element.textContent` replaces the text inside an element; unlike `innerHTML` it treats the value as plain text and does not interpret HTML tags, which makes it the safer choice.
- `element.style.color`, `style.backgroundColor`, `style.padding` and `style.borderRadius` set inline CSS on the element, which overrides the styles from a stylesheet.
- `element.setAttribute("title", value)` adds or updates an attribute on the element.
- The script is placed at the end of the body so that all the elements already exist in the DOM when the script runs.
- The changes exist only in the DOM in memory, so refreshing the page restores the original HTML.

The basic flow of the program is:

The page loads and the browser builds the DOM → the three elements are selected and stored in constants → the user clicks the Update Page button → the click handler runs → the heading text and the paragraph text are replaced → the heading colour, background colour, padding and border radius are set → a title attribute is added to the heading → the browser repaints the page immediately, without reloading it.

5. ALGORITHM / PROCEDURE

- Create the file `dom.html` with an `<h1>` having `id="title"`, a `<p>` having `id="description"` and a `<button>` having `id="updateBtn"`.
- Write the JavaScript inside a `<script>` tag placed at the end of the body, so that the elements already exist when the script runs.
- Select the heading, the paragraph and the button using `document.getElementById()` and store them in constants.
- Attach a click event listener to the button using `addEventListener()`.
- Inside the handler, change the content by setting `title.textContent` and `description.textContent` to their new values.
- Change the style by setting `title.style.color` to white, `backgroundColor` to teal, `padding` to 12px and `borderRadius` to 6px.
- Change the attribute by calling `title.setAttribute("title", "Heading text updated via JavaScript")`.
- Save the file and open it in a browser; note the original content, then click the button and observe the content and style change.
- Open the browser console and inspect the heading before and after the click to confirm the inline style and the title attribute.
- Click the button a second time and then refresh the page to observe that the DOM is rebuilt from the HTML file.

6. PROGRAM

dom.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Manipulation Demo</title>
</head>

<body>

  <h1 id="title">Static Heading</h1>

  <p id="description">
    This paragraph has not been touched by JavaScript yet.
  </p>

  <button id="updateBtn">
    Update Page
  </button>

  <script>
    const title = document.getElementById("title");
    const description = document.getElementById("description");
    const updateBtn = document.getElementById("updateBtn");

    updateBtn.addEventListener("click", function () {

      // Change content
      title.textContent = "Heading Updated Successfully";

      description.textContent =
        "JavaScript just changed this text using the DOM.";

      // Change style
      title.style.color = "white";
      title.style.backgroundColor = "teal";
      title.style.padding = "12px";
      title.style.borderRadius = "6px";

      // Change an attribute
      title.setAttribute("title", "Heading text updated via JavaScript");
    });
  </script>

</body>
</html>
```

7. CODE EXPLANATION

Code	Explanation
document.getElementById("title")	Selects the <h1> node from the DOM so that JavaScript can modify it.
document.getElementById("description")	Selects the paragraph whose text will be replaced.
document.getElementById("updateBtn")	Selects the button that triggers all the changes.
addEventListener("click", function () { ... })	Runs the given function whenever the button is clicked, instead of when the page loads.
title.textContent = "Heading Updated Successfully"	Changes the content: replaces the text inside the heading.
description.textContent = "..."	Changes the content of the paragraph in the same way.

Code	Explanation
<code>title.style.color = "white"</code>	Changes the style: applies an inline text colour to the heading.
<code>title.style.backgroundColor = "teal"</code>	Applies a background colour; the CSS property background-color becomes backgroundColor in JavaScript.
<code>title.style.padding / borderRadius</code>	Adds inline spacing inside the heading and rounds its corners.
<code>title.setAttribute("title", "...")</code>	Changes the attributes: adds a title attribute, which the browser shows as a tooltip on hover.

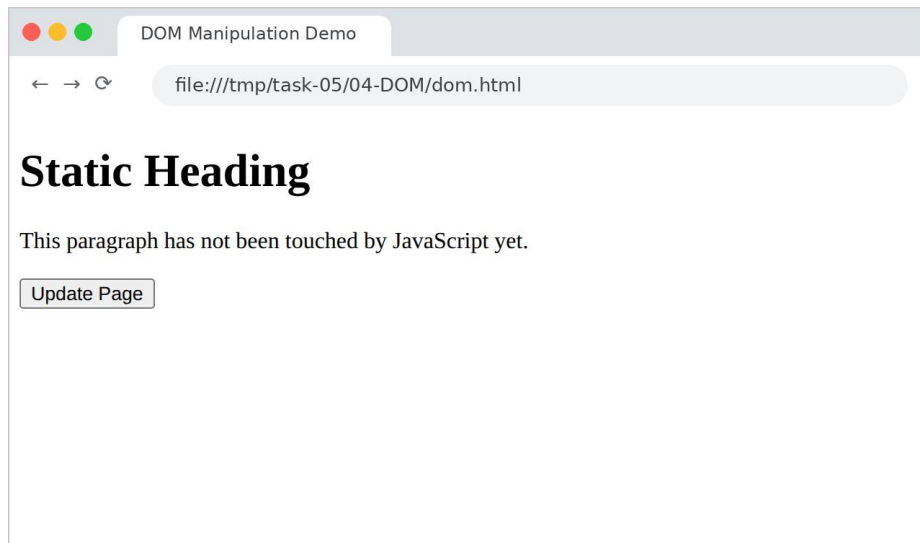
8. TEST CASES AND EXECUTION DATA

The page was opened in a browser and tested before and after the click, including an inspection of the element in the browser console and a check that the change is not permanent. The results were recorded below.

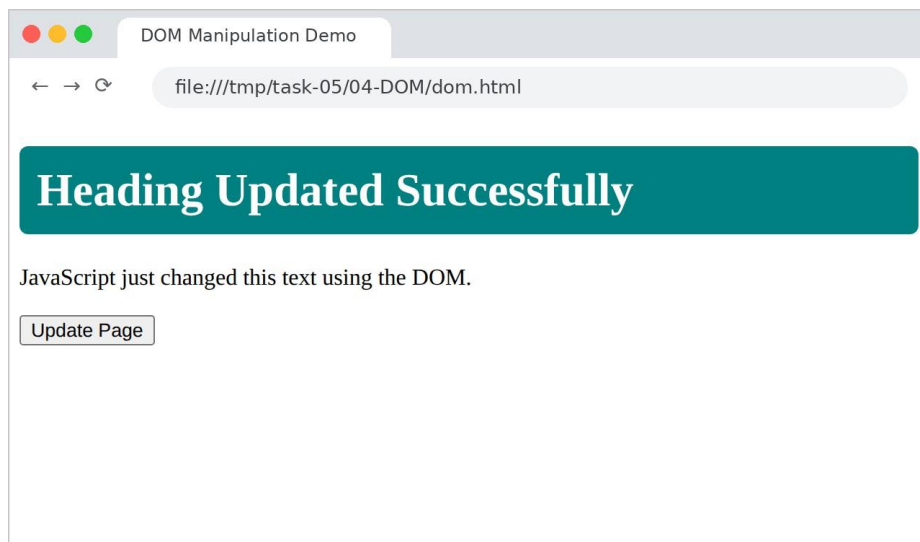
Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open dom.html in a browser	The heading "Static Heading", the original paragraph and the Update Page button are displayed in the default style	Original page displayed	Pass
TC-02	Click the "Update Page" button	The heading and paragraph text change, and the heading turns white on a teal background with padding and rounded corners	Content and style changed without reloading the page	Pass
TC-03	Inspect the heading in the console before and after the click	Before: no title attribute and no inline style. After: the title attribute and the inline style are present on the <h1>	getAttribute("title") returned null before and "Heading text updated via JavaScript" after; inline style present	Pass
TC-04	Click the button a second time	The same values are set again, so the page stays as it is and no error occurs	Output unchanged, no error	Pass
TC-05	Refresh the page after clicking	The DOM is rebuilt from the HTML file and the original content returns	Original content restored	Pass

9. OUTPUT

TC-01 – Original page before the button is clicked



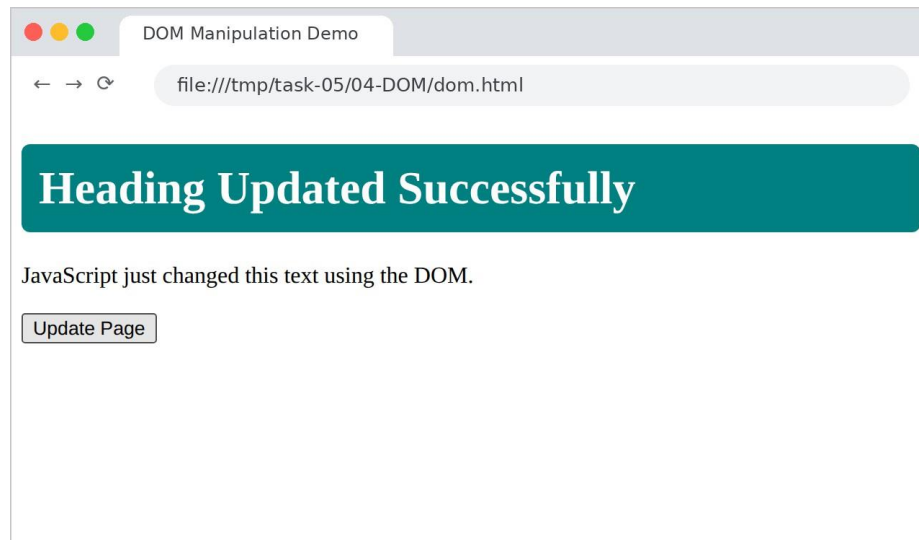
TC-02 – Content and style changed after clicking Update Page



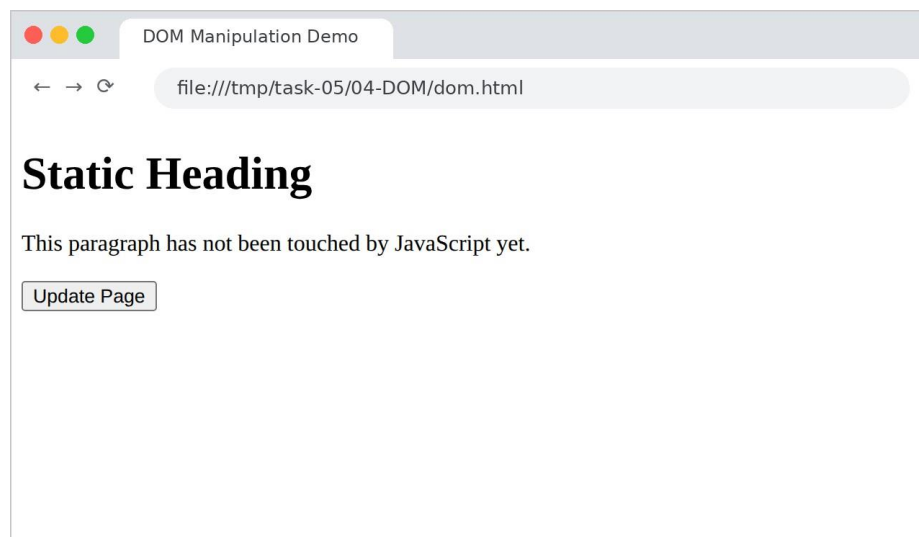
TC-03 – Element inspected in the console before and after the click



TC-04 – Clicking a second time leaves the output unchanged



TC-05 – After refreshing, the original page is restored



10. RESULT

Thus, the Document Object Model was explained and demonstrated successfully. On clicking the Update Page button, JavaScript changed the content of the heading and the paragraph using `textContent`, the style of the heading using the `style` object and an attribute of the heading using `setAttribute()`, and the browser repainted the page immediately without reloading it. Inspecting the element confirmed that the inline style and the title attribute had been added in the DOM. Clicking the button again produced no visible change because the same values were set, and refreshing the page restored the original content, because the changes are held only in memory and the DOM is rebuilt from the HTML file on every load.